



KE&M II: Ontology Design Tips

Hutchatai Chanlekha

Department of Computer Engineering, Kasetsart University





Defining classes and a class hierarchy

- There is no single correct class hierarchy for any given domain.
- The hierarchy depends on
 - the possible uses of the ontology
 - the level of the detail that is necessary for the application
 - personal preferences
 - sometimes including requirements for compatibility with other models
- However, there are several guidelines to keep in mind when developing a class hierarchy.



Ensuring that the class hierarchy is correct

- “is-a” relation

- The class hierarchy represents an “is-a” relation
- Class A is a subclass of B if every instance of A is also an instance of B.
Dog is a subclass of animal → all instances of dog are also animal
- Another way to think of the taxonomic relation is as a “kind-of” relation.

Dog is a kind of animal

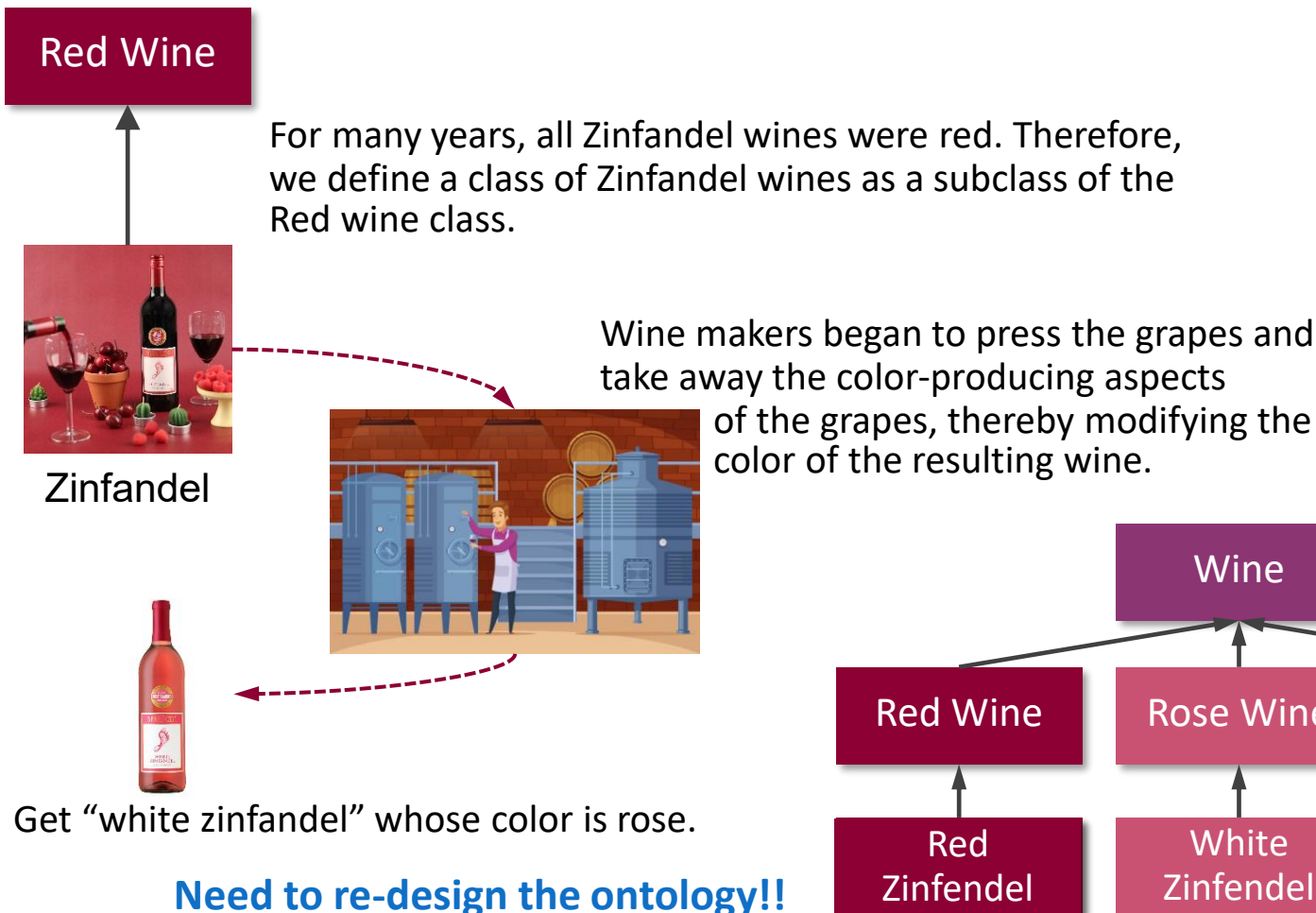
A subclass of a class represents a concept that is a “kind of” the concept that the superclass represents

- Transitivity of the hierarchical relations

- A subclass relationship is transitive:
If C is a subclass of B and B is a subclass of A, then C is a subclass of A
- Example:
 - Dog is a subclass of Mammal, Mammal is a subclass of Animal,
 - Then Dog is a subclass of Animal.

Evolution of a class hierarchy

- Maintaining a consistent class hierarchy may become challenging as domains evolve.
- Example:



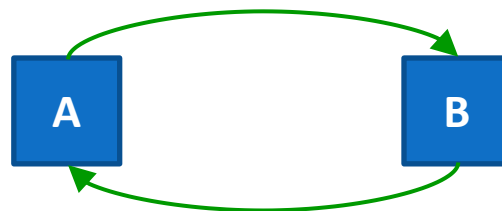


Classes and their names

- *Classes represent concepts in the domain and not the words that denote these concepts*
 - The name of a class may change if we choose a different terminology, but the class itself represents the objective reality in the world.
 - For example, we may create a class Shrimps, and then rename it to Prawns—the class still represents the same concept.
- *Synonyms for the same concept do not represent different classes*
 - Many systems allow associating a list of synonyms, translations, or presentation names with a class.
 - If a system does not allow this associations, synonyms could always be listed in the class documentation.
 - Synonyms are just different names for a concept or a term. Therefore, we should not have synonyms referred to different classes.
 - For example, we should not have a class called Dog and a class called Canine
 - Rather, there is one class, named either Dog or Canine.

Avoiding class cycles

- Avoid cycles in the class hierarchy.

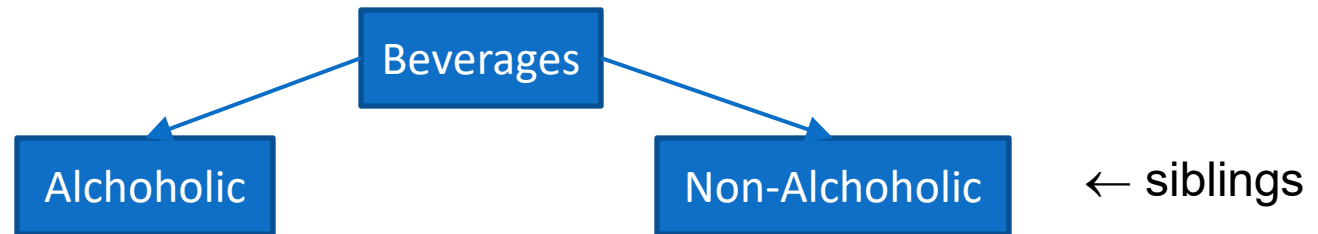


- Cycle happened in a hierarchy when some class A has a subclass B and at the same time B is a superclass of A.
- Creating such a cycle in a hierarchy amounts to declaring that the classes A and B are equivalent:
 - since B is a subclass of A, all B's instances must be instances of the class A. Since A is a subclass of B, all A's instances must also be instances of the class B.



Analyzing siblings in a class hierarchy

- Siblings in the hierarchy are classes that are direct subclasses of the same class



- Keep in mind
 - All the siblings in the hierarchy must be at the same level of generality.
 - For example, “Coffee” and “Alcoholic” shouldn’t be subclasses of the same class (i.e. Beverages) because Alcoholic is more general than Coffee.
 - Siblings should represent concepts that fall “along the same line”
- Remark: except for siblings that are children of root node
 - Reason: The concepts at the root of the hierarchy normally represent major divisions of the domain

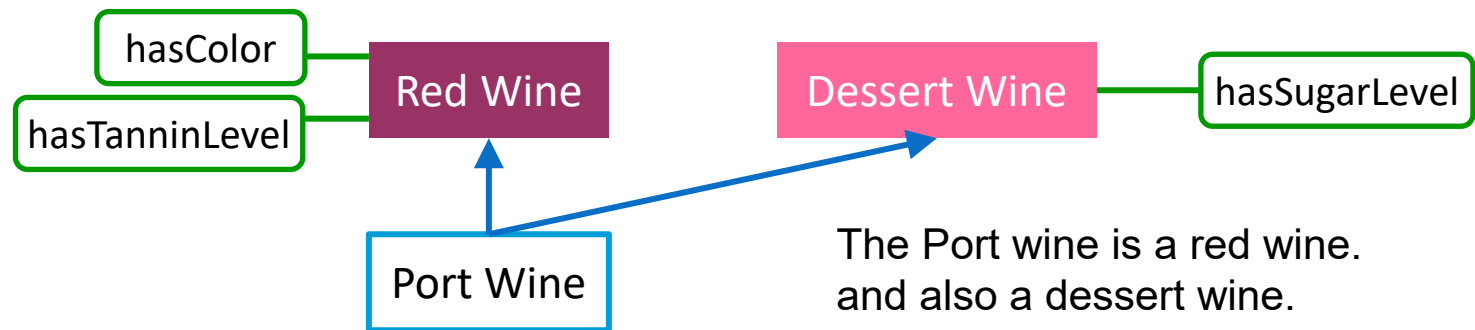


Appropriate number of direct subclasses

- There are no hard rules for the number of direct subclasses that a class should have.
- However, many well structured ontologies have between two and a dozen direct subclasses.
- Therefore, the following two guidelines:
 - If a class has only one direct subclass, there may be a modeling problem, or the ontology is not complete.
 - If there are more than a dozen subclasses for a given class, then additional intermediate categories may be necessary.
 - For example, instead of adding all wine under Wine class, you can add intermediate categories, such as “White wine”, “Red wine”, “rosé wine”, and “sparkling wine”

Multiple Inheritance

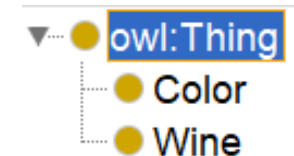
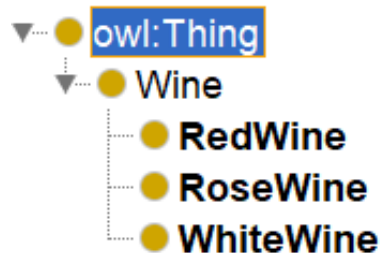
- Most knowledge-representation systems allow multiple inheritance in the class hierarchy:
 - A class can be a subclass of several classes.
- For example: Suppose in your ontology, you have only Red wine class, but one day, you want to create a separate class of dessert wines, the Dessert wine class.



- All instances of the Port class will be instances of both the Red wine class and the Dessert wine class.
- !! Be careful when there is a conflict between the properties of its parents

When to introduce a new class

- One of the hardest decisions to make during modeling is when to introduce a new class or when to represent a distinction through different property values.
- For example:
 - For Wine ontology, if we want to represent a color of wine, what should we do?



- Represent in a form of subclass
- Create *Color* class with instances: red, white, rose
- Then create property *hasColor* for Wine class



When to introduce a new class

- Keep in mind
 - Ontology becomes hard to navigate both when
 - Extremely nested hierarchy with many extraneous classes
 - Very flat hierarchy that has not enough number of classes with too much information encoded in property slots.
 - When defining a class hierarchy, the goal is to strike a balance between creating new classes useful for class organization and creating too many classes.



When to introduce new class

- Several rules of thumb that help decide when to introduce new classes in a hierarchy.
 - Subclasses of a class usually
 - have additional properties that the superclass does not have,
 - Ex. Red wines can have different levels of tannin, whereas this property is not used to describe wines in general.
 - Or have restrictions different from those of the superclass,
 - Ex. The value for the sugar-level property of the Dessert Wine is HIGH, whereas it is not true of its superclass.
 - Or participate in different relationships than the superclass
- In practical terms, each subclass should either have new properties added to it, or have new property values defined, or override some facets for the inherited properties.



When to introduce new class

- Sometimes it may be useful to create new classes even if they do not introduce any new properties.
 - For example, classes in terminological hierarchies
- Example: an ontology of an electronic medical-record system may include a classification of various diseases.
 - This classification may be just a hierarchy of terms without properties. In that case, it is still useful to organize the terms in a hierarchy rather than a flat list, because:
 - It will allow easier exploration and navigation
 - It will enable a doctor to easily choose a level of generality of the term that is appropriate for the situation.
- Another reason to introduce new classes without any new properties is when the domain experts commonly make a distinction among these classes.



When to introduce new class

- Another reason to introduce new classes without any new properties is when the domain experts commonly make a distinction among these classes.
- Since ontologies are used to facilitate communication among domain experts and between domain experts and knowledge-based systems, we would like to reflect the expert's view of the domain in the ontology.



A new class or a property value?

- Deciding whether to model a specific distinction as a property value or as a set of classes depends on the scope of the domain and the task at hand.
 - The answer usually lies in the scope that we defined for the ontology.
 - How important the concept is in our domain?
- For example: White wine, Red wine, Rosé wine
 - Should we create as subclasses of wine, or do we simply create a class Wine and fill in different values for the property color?
 - For a domain model used in a factory producing wine labels, rules for wine labels of any color are the same and the distinction is not very important. Thus, it's not necessary to separate class for wine.
 - For the representation of wine, food, and their appropriate combinations, a red wine is very different from a white wine: it is paired with different foods, has different properties, and so on. Thus, we create a separate class for wine.



More example:

- Consider the human-anatomy ontology.
- When we represent ribs,
 - Should we create a class for each of the “1st left rib”, “2nd left rib”, and so on?
 - Should we have a class Rib with properties for the order and the lateral position (left-right)?
- Answer
 - If the information about each of the ribs represented in our ontology is significantly different, then we should create a class for each ribs.
 - For example, if we want to represent detail adjacency and location information, which is different for each rib, as well as specific functions that each rib plays and organs it protects, then we want the classes.
 - If we are modeling anatomy at a slightly lesser level of generality, and all ribs are very similar as far as our potential applications are concerned, we can have just the class Rib, with two properties: lateral position, and order.



Summary: New class vs. Property value

- If the concepts with different property values become restrictions for different properties with other classes
- Then we should create a new class for the distinction.
- Otherwise, we represent the distinction in a property value.

- If a distinction is important in the domain and we think of the objects with different values for the distinction as different kinds of objects,
- Then we should create a new class for the distinction.



When categorizing class

- Keep in mind that a class to which an individual instance belongs should not change often
 - Usually when we use extrinsic rather than intrinsic properties of concepts to differentiate among classes, instances of those classes will have to migrate often from one class to another
 - For example, Chilled wine
 - “Chilled wine” should not be a class in an ontology describing wine bottles in a restaurant.
 - Why?
 - The property chilled should be an attribute of wine in a bottle since an instance of Chilled wine can easily cease being an instance of this class.



An instance or a class?

- Deciding whether a particular concept is a class in an ontology or an individual instance depends on what the potential applications of the ontology are.
- The level of granularity is in turn determined by a potential application of the ontology.
 - What are the most specific items that are going to be represented in the knowledge base?
 - The most specific entities that will constitute answers to the competency questions are very good candidates for individuals.

Individual instances are the most specific entities represented in a knowledge base.



Example: Class vs. Instance

- if we are only going to talk about recommending soft drink
 - We will not be interested in the specific physical bottles of drink.
 - Therefore, such terms as Cola are probably going to be the most specific terms we use.
 - Therefore, Cola would be an instance in the knowledge base.
- On the other hand, if we would like to maintain an inventory of soft drink in the convenient store,
 - Individual bottles of each soft drink may become individual instances in our knowledge base.



Limiting the scope

- The following set of rules is always helpful in deciding when an ontology definition is complete:
 - The ontology should NOT contain all the possible information about the domain, **should contain just what you need**:
 - Do not need to specialize (or generalize) more than you need for your application (at most one extra level each way).
 - Similarly, the ontology should NOT contain all the possible properties of and distinctions among classes in the hierarchy more than you need for your application



Naming Convention

Define a naming convention for classes and properties and adhere to it.

- Defining naming conventions for concepts in an ontology and strictly adhering to these conventions not only makes the ontology easier to understand but also helps avoid some common modeling mistakes.
- There are many alternatives in naming concepts.
- Often there is no particular reason to choose one or another alternative.



Naming Convention

- Before defining naming convention, check your KB system first
 - Does the system have the same name space for classes, properties, and instances? Does the system allow having a class and a property with the same name?
 - Is the system case-sensitive? Does the system treat the names that differ only in case as different names (such as Winery and winery)?
 - What delimiters does the system allow in the names? Can names contain spaces, underscores, commas, asterisks, and so on?
- These features will affect your choice of naming convention



Naming: Capitalization and delimiters

- Improve the readability of an ontology if we use consistent capitalization for concept names.
 - For example,
 - Uppercase letter for the first letter of class names
 - Lowercase letter for the first letter of property names (assuming the system is case-sensitive).
- When a concept name contains more than one word, we need to delimit the words.
 - Use Space: `Meal course`
 - Run the words together and capitalize each new word: `MealCourse`
 - Use an underscore or dash or other delimiters in the name:
`Meal_Course`, `Meal_course`, `Meal-Course`, `Meal-course`, **etc.**



Naming: Singular or plural

- Singular for class names is used more often in practice
- However, it could be more natural for some designers to call the class with plural form (Wines) rather than single form (Wine).
- No alternative is better or worse than the other (although).
- Whatever the choice, it should be consistent throughout the whole ontology.



Naming: Prefix and suffix conventions

- Some methodologies suggest using prefix and suffix conventions in the names to distinguish between classes and properties.
- Two common practices are to add a prefix has- or a suffix -of to property names.
 - For example: describing the maker and winery of a wine
 - Name property as “has-maker” and “has-winery” if we chose the has- convention.
 - Or name property “maker-of” and “winery-of” if we chose the of- convention.
- This approach allows anyone looking at a term to determine immediately if the term is a class or a property.



Other naming considerations

- Do not add strings such as “class”, “property”, “slot”, and so on to concept names.
- It is usually a good idea to avoid abbreviations in concept names
 - For example, use Cabernet Sauvignon rather than Cab
- Names of direct subclasses of a class should either all include or not include the name of the superclass.
 - For example, if we are creating two subclasses of the Wine class to represent red and white wines, the two subclass names should be
 - ✓ • Red Wine and White Wine, or
 - ✓ • Red and White
 - ✗ • but not Red Wine and White.



Tip to help organizing class hierarchy

- When you have to model unfamiliar concepts, thesaurus or taxonomy could be great help
 - Hypernym: for more general concept
 - Hyponym: for more specific concept
- For example: Planet
 - From <https://en.wiktionary.org/wiki/Thesaurus:planet>
 - Hypernym: celestial body
 - Hyponym: giant planet, major planet, minor planet, terrestrial planet, etc.



Reference

- Slides based on
 - **Ontology Development 101: A Guide to Creating Your First Ontology**
 - Natalya F. Noy and Deborah L. McGuinness
 - https://protege.stanford.edu/publications/ontology_development/ontology101.pdf
 - https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html